

# homework4

2026-09-20

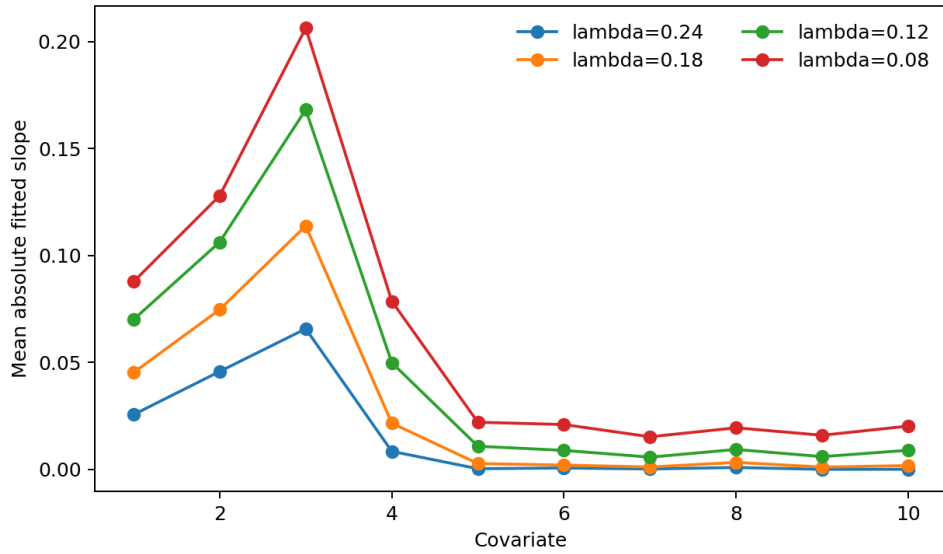
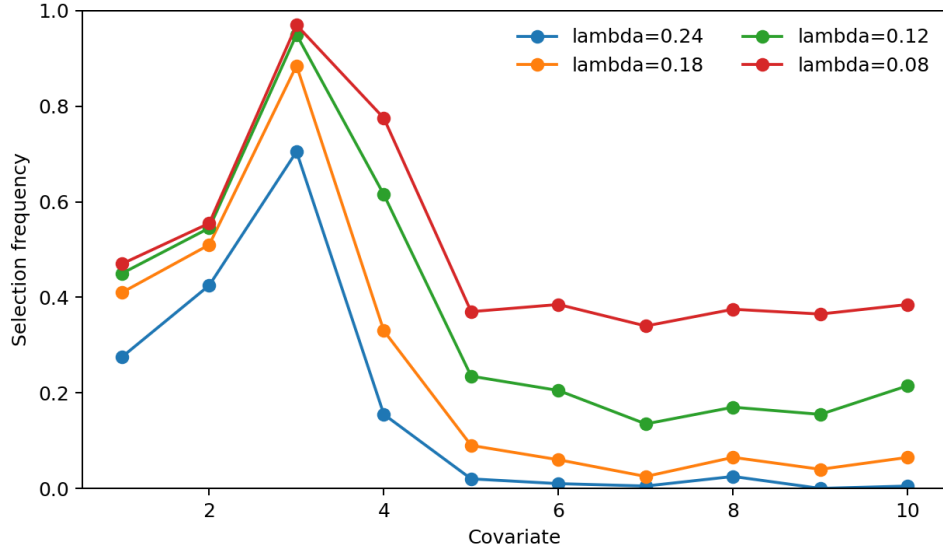
## Homework 4

### Question 1

a) The selection frequencies and mean absolute fitted slopes are shown below.

| Covariate | $\lambda = 0.24$ | $\lambda = 0.18$ | $\lambda = 0.12$ | $\lambda = 0.08$ |
|-----------|------------------|------------------|------------------|------------------|
| X1        | 0.275            | 0.410            | 0.450            | 0.470            |
| X2        | 0.425            | 0.510            | 0.545            | 0.555            |
| X3        | 0.705            | 0.885            | 0.950            | 0.970            |
| X4        | 0.155            | 0.330            | 0.615            | 0.775            |
| X5        | 0.020            | 0.090            | 0.235            | 0.370            |
| X6        | 0.010            | 0.060            | 0.205            | 0.385            |
| X7        | 0.005            | 0.025            | 0.135            | 0.340            |
| X8        | 0.025            | 0.065            | 0.170            | 0.375            |
| X9        | 0.000            | 0.040            | 0.155            | 0.365            |
| X10       | 0.005            | 0.065            | 0.215            | 0.385            |

| Covariate | $\lambda = 0.24$ | $\lambda = 0.18$ | $\lambda = 0.12$ | $\lambda = 0.08$ |
|-----------|------------------|------------------|------------------|------------------|
| X1        | 0.026            | 0.045            | 0.070            | 0.088            |
| X2        | 0.046            | 0.075            | 0.106            | 0.128            |
| X3        | 0.066            | 0.114            | 0.168            | 0.206            |
| X4        | 0.008            | 0.022            | 0.050            | 0.079            |
| X5        | 0.000            | 0.003            | 0.011            | 0.022            |
| X6        | 0.001            | 0.002            | 0.009            | 0.021            |
| X7        | 0.000            | 0.001            | 0.006            | 0.015            |
| X8        | 0.001            | 0.003            | 0.009            | 0.019            |
| X9        | 0.000            | 0.001            | 0.006            | 0.016            |
| X10       | 0.000            | 0.002            | 0.009            | 0.020            |



As  $\lambda$  increases, coefficients are shrunk more strongly, so both fitted magnitudes and selection frequencies decrease. The weakest signals and noise variables drop out first.

b)

| $\lambda$ | X1 only | X2 only | Both  | Neither |
|-----------|---------|---------|-------|---------|
| 0.24      | 0.270   | 0.420   | 0.005 | 0.305   |
| 0.18      | 0.400   | 0.500   | 0.010 | 0.090   |
| 0.12      | 0.430   | 0.525   | 0.020 | 0.025   |
| 0.08      | 0.440   | 0.525   | 0.030 | 0.005   |

$X_1$  and  $X_2$  have similar marginal selection rates because they contain almost the same information and have the same population coefficient. Lasso usually keeps only one of them because either variable can represent nearly the same signal, and stronger penalties increase the chance that neither is kept.

c)

| $\lambda$ | X3    | X4    | Noise avg. |
|-----------|-------|-------|------------|
| 0.24      | 0.705 | 0.155 | 0.011      |
| 0.18      | 0.885 | 0.330 | 0.057      |
| 0.12      | 0.950 | 0.615 | 0.186      |
| 0.08      | 0.970 | 0.775 | 0.370      |

$X_3$  is selected most often, followed by the weaker  $X_4$ , while the noise average is much lower at larger penalties. With highly correlated predictors, a zero estimate can simply mean another variable replaced it, and a nonzero estimate does not prove that variable is uniquely important.

```
import numpy as np
from sklearn.linear_model import Lasso

rng = np.random.default_rng(43246)
n, p, R = 120, 10, 200
b = np.array([.15, .15, .30, .15, 0, 0, 0, 0, 0, 0])
lams = np.array([.24, .18, .12, .08])
bhat = np.zeros((R, p, len(lams)))

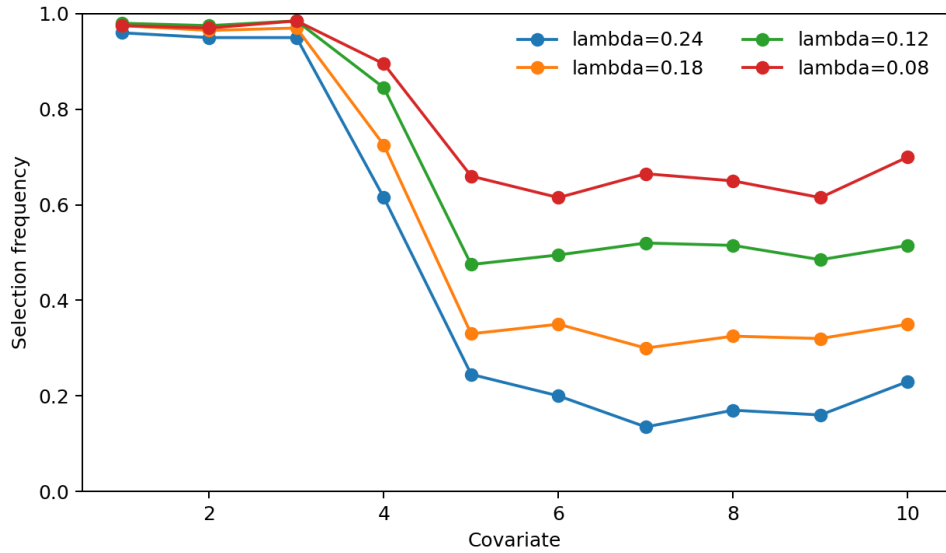
for k in range(R):
    U = rng.normal(size=(n, p))
    X = U.copy()
    X[:, 1] = .999*U[:, 0] + np.sqrt(1-.999**2)*U[:, 1]
    y = X@b + rng.normal(size=n)
    Xc = X - X.mean(0)
    s = np.sqrt(np.mean(Xc**2, axis=0))
    Xs, yc = Xc/s, y-y.mean()
    fit = Lasso(fit_intercept=False, warm_start=True,
               tol=1e-10, max_iter=100000)
    for m, lam in enumerate(lams):
        fit.set_params(alpha=lam).fit(Xs, yc)
        bhat[k, :, m] = fit.coef_/s

sel = np.abs(bhat) > 1e-8
selection_freq = sel.mean(0)
mean_abs_slope = np.abs(bhat).mean(0)
```

## Question 2

a)

| Covariate | $\lambda = 0.24$ | $\lambda = 0.18$ | $\lambda = 0.12$ | $\lambda = 0.08$ |
|-----------|------------------|------------------|------------------|------------------|
| X1        | 0.960            | 0.975            | 0.980            | 0.975            |
| X2        | 0.950            | 0.965            | 0.975            | 0.970            |
| X3        | 0.950            | 0.970            | 0.985            | 0.985            |
| X4        | 0.615            | 0.725            | 0.845            | 0.895            |
| X5        | 0.245            | 0.330            | 0.475            | 0.660            |
| X6        | 0.200            | 0.350            | 0.495            | 0.615            |
| X7        | 0.135            | 0.300            | 0.520            | 0.665            |
| X8        | 0.170            | 0.325            | 0.515            | 0.650            |
| X9        | 0.160            | 0.320            | 0.485            | 0.615            |
| X10       | 0.230            | 0.350            | 0.515            | 0.700            |



Elastic net raises the selection frequencies relative to lasso, with the largest increases for  $X_1$  and  $X_2$ . The weaker signal and noise variables also enter more often because the  $\ell_1$  part of the penalty is smaller.

b)

| $\lambda$ | X1 only | X2 only | Both  | Neither |
|-----------|---------|---------|-------|---------|
| 0.24      | 0.025   | 0.015   | 0.935 | 0.025   |
| 0.18      | 0.020   | 0.010   | 0.955 | 0.015   |
| 0.12      | 0.020   | 0.015   | 0.960 | 0.005   |
| 0.08      | 0.025   | 0.020   | 0.950 | 0.005   |

The pair is now selected together in about 94%-96% of repetitions. The squared  $\ell_2$  term favors sharing weight across highly correlated predictors instead of putting nearly all of the coefficient on only one.

c)

| $\lambda$ | X3    | X4    | EN noise avg. | Lasso noise avg. |
|-----------|-------|-------|---------------|------------------|
| 0.24      | 0.950 | 0.615 | 0.190         | 0.011            |
| 0.18      | 0.970 | 0.725 | 0.329         | 0.057            |
| 0.12      | 0.985 | 0.845 | 0.501         | 0.186            |
| 0.08      | 0.985 | 0.895 | 0.651         | 0.370            |

Noise variables are still screened more strongly as  $\lambda$  increases, but less aggressively than with lasso because elastic net uses only half of  $\lambda$  as its  $\ell_1$  threshold. At  $\lambda = 0.24$  and  $0.18$ , both correlated variables are usually selected while most noise variables remain excluded.

```
from sklearn.linear_model import ElasticNet

rng = np.random.default_rng(43246)
bhat_en = np.zeros_like(bhat)
for k in range(R):
    U = rng.normal(size=(n,p))
    X = U.copy()
```

```

X[:,1] = .999*U[:,0] + np.sqrt(1-.999**2)*U[:,1]
y = X@b + rng.normal(size=n)
Xc = X - X.mean(0)
s = np.sqrt(np.mean(Xc**2, axis=0))
Xs, yc = Xc/s, y-y.mean()
fit = ElasticNet(l1_ratio=.5, fit_intercept=False,
                 warm_start=True, tol=1e-10,
                 max_iter=100000)
for m, lam in enumerate(lams):
    fit.set_params(alpha=lam).fit(Xs, yc)
    bhat_en[k,:,m] = fit.coef_/s

sel_en = np.abs(bhat_en) > 1e-8
selection_freq_en = sel_en.mean(0)

```

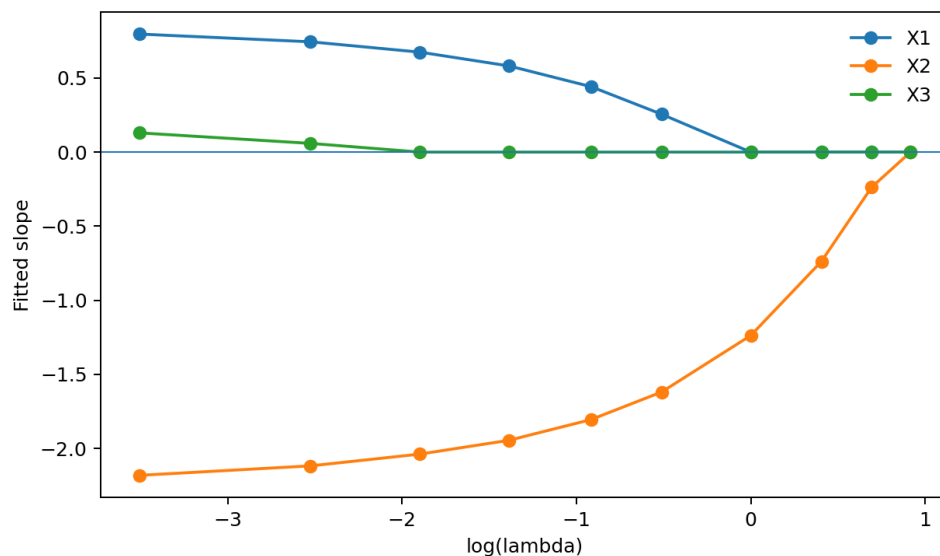
### Question 3

a)

| $\lambda$ | $\hat{\beta}_1$ | $\hat{\beta}_2$ | $\hat{\beta}_3$ | Cycles |
|-----------|-----------------|-----------------|-----------------|--------|
| 2.50      | 0.0000          | -0.0000         | -0.0000         | 1      |
| 2.00      | 0.0000          | -0.2373         | -0.0000         | 2      |
| 1.50      | 0.0000          | -0.7373         | -0.0000         | 2      |
| 1.00      | 0.0000          | -1.2373         | -0.0000         | 2      |
| 0.60      | 0.2553          | -1.6183         | -0.0000         | 5      |
| 0.40      | 0.4416          | -1.8046         | 0.0000          | 5      |
| 0.25      | 0.5812          | -1.9442         | 0.0000          | 5      |
| 0.15      | 0.6743          | -2.0373         | 0.0000          | 5      |
| 0.08      | 0.7441          | -2.1168         | 0.0587          | 7      |
| 0.03      | 0.7963          | -2.1807         | 0.1298          | 7      |

All fits converged.  $\lambda_{\max} = 2.2373$ , so the fit is all zero at  $\lambda = 2.5$ , the only grid value at or above  $\lambda_{\max}$ .

b)



$X_2$  enters first, then  $X_1$ , and  $X_3$  enters only at the two smallest penalties. This matches the true signal sizes because  $|-2| > |1| > 0$ .

c) At  $\lambda = 0.03$ , the lasso fit is  $(0.7963, -2.1807, 0.1298)$ , compared with least squares  $(0.8276, -2.2191, 0.1724)$ . They are close because the penalty is small, but lasso still applies some shrinkage, so the estimates are not identical.

```
rng = np.random.default_rng(43247)
X = rng.normal(size=(100,3))
y = 1.5 + X[:,0] - 2*X[:,1] + rng.normal(size=100)
Xc = X-X.mean(0)
Xs = Xc/np.sqrt(np.mean(Xc**2, axis=0))
yc = y-y.mean()
lams = np.array([2.5,2,1.5,1,.6,.4,.25,.15,.08,.03])

def soft(a, lam):
    return np.sign(a)*max(abs(a)-lam, 0)

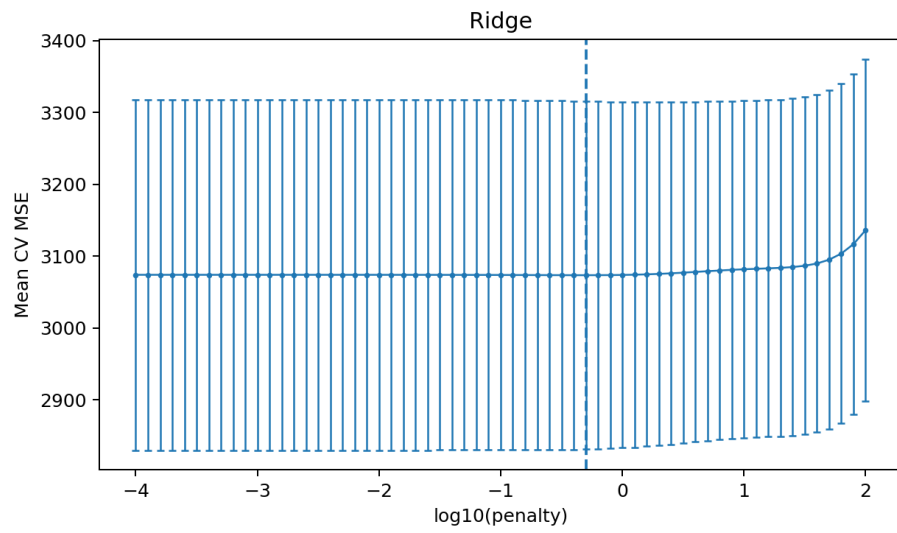
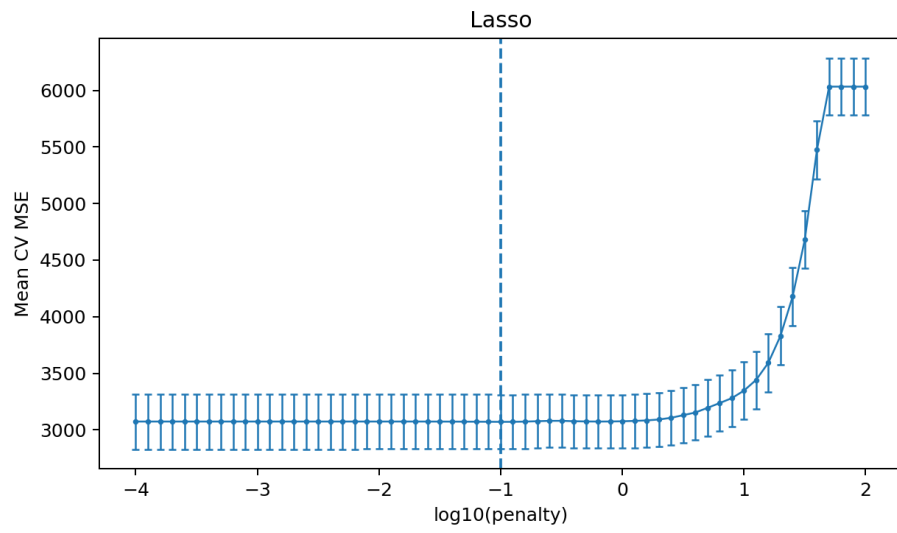
def lasso_path(X, y, lams, tol=1e-8):
    beta = np.zeros(X.shape[1]); path=[]; cycles=[]
    for lam in lams:
        for it in range(1,1001):
            old = beta.copy()
            for j in range(X.shape[1]):
                r = y-X@beta+X[:,j]*beta[j]
                beta[j] = soft(np.mean(X[:,j]*r), lam)
            if np.max(np.abs(beta-old)) < tol: break
        path.append(beta.copy()); cycles.append(it)
    return np.array(path), np.array(cycles)

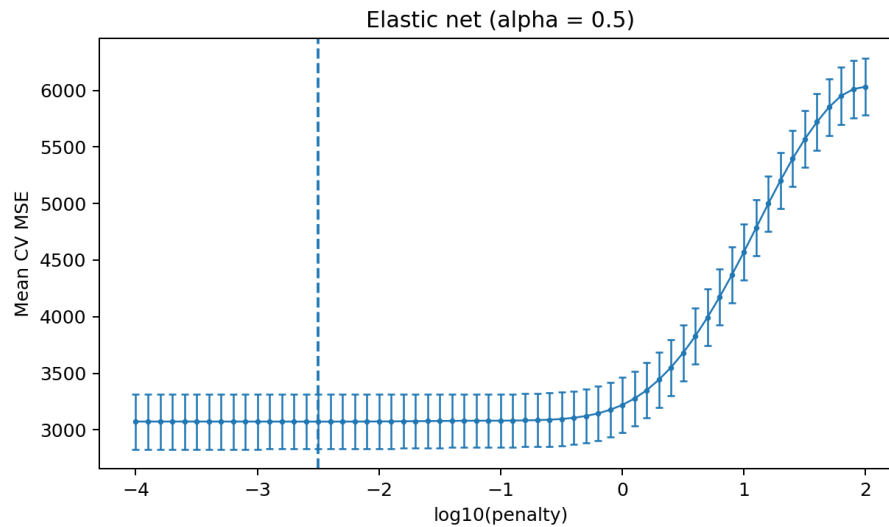
path, cycles = lasso_path(Xs, yc, lams)
lambda_max = np.max(np.abs(np.mean(Xs*yc[:,None], axis=0)))
ls = np.linalg.lstsq(Xs, yc, rcond=None)[0]
```

## Question 4

| Method      | Penalty | CV MSE | CV SE |
|-------------|---------|--------|-------|
| Lasso       | 0.1000  | 3071.2 | 240.4 |
| Ridge       | 0.5012  | 3073.5 | 242.1 |
| Elastic net | 0.0032  | 3073.4 | 242.1 |

Selected method: **Lasso**. Selected penalty: 0.1000. Final test MSE: 2732.4.





```
import pandas as pd
from sklearn.linear_model import Lasso, Ridge, ElasticNet
from sklearn.model_selection import GridSearchCV
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler

D = pd.read_csv("data/diabetes.csv")
S = pd.read_csv("data/diabetes-split-folds.csv")
D.insert(0, "row_id", np.arange(1, len(D)+1))
A = S.merge(D, on="row_id", validate="one_to_one")
features = ["age", "sex", "bmi", "bp", "s1", "s2", "s3", "s4", "s5", "s6"]
train = A.split.eq("train")
Xtr, ytr = A.loc[train, features].to_numpy(), A.loc[train, "y"].to_numpy()
Xte, yte = A.loc[~train, features].to_numpy(), A.loc[~train, "y"].to_numpy()
fold = A.loc[train, "cv_fold"].to_numpy()
cv = [(np.flatnonzero(fold!=f), np.flatnonzero(fold==f))
      for f in np.unique(fold)]

grid = np.logspace(-4, 2, 61)
models = {"Lasso": Lasso(max_iter=100000, tol=1e-10),
          "Ridge": Ridge(),
          "Elastic net": ElasticNet(l1_ratio=.5, max_iter=100000, tol=1e-10)}
results = {}
for name, model in models.items():
    pipe = Pipeline([("scale", StandardScaler()), ("model", model)])
    gs = GridSearchCV(pipe, {"model__alpha": grid},
                      scoring="neg_mean_squared_error", cv=cv, refit=True)
    gs.fit(Xtr, ytr)
    results[name] = gs

winner = min(results, key=lambda k: -results[k].best_score_)
best = results[winner]
test_mse = np.mean((yte-best.predict(Xte))**2)
```